ENTREGA PROYECTO FINAL CODERHOUSE - SQL
Informe Base de Datos Relacional
–
Cheers LK

Mora Coghlan
2026

Índice

1. Introducción ---------- 3
2. Objetivos del proyecto ---------- 4
3. Descripción del modelo de negocio —----------5
4. Diagrama Entidad-Relación (ER) —----------7
5. Descripción de tablas —----------8
6. Inserción de datos de prueba —----------16
7. Vistas —----------16
8. Funciones y procedimientos —----------17
9. Triggers —----------19
11. Informes generados —----------21
12. Orden de ejecución —----------26
13. Herramientas y tecnología utilizada —----------27
14. Repositorios y links de acceso —----------27

1. Introducción

En este proyecto, se desarrolla una base de datos relacional para el emprendimiento **Cheers LK**, dedicado a la venta de buzos personalizados. La elección de este proyecto surge de la experiencia cercana con el negocio, lo que permite una estructura realista y detallada de la base de datos.

El objetivo principal es implementar una base de datos que registre clientes, pedidos, productos, proveedores y pagos, garantizando integridad, consistencia y posibilidad de análisis de información.

2. Objetivos del proyecto

- Crear una base de datos relacional basada en un modelo de negocio realista.
- Implementar objetos que permitan el mantenimiento y operación eficiente de la base de datos.
- Desarrollar consultas SQL que faciliten la generación de informes y análisis de datos.

3. Descripción del modelo de negocio

Cheers LK es un emprendimiento dedicado a la venta de **buzos personalizados**, enfocado en ofrecer productos únicos adaptados a las preferencias de cada cliente. La elección de esta temática surge de la cercanía con el negocio, ya que dos integrantes del grupo poseen esta experiencia, lo que permite disponer de información realista y detallada para la estructuración de la base de datos.

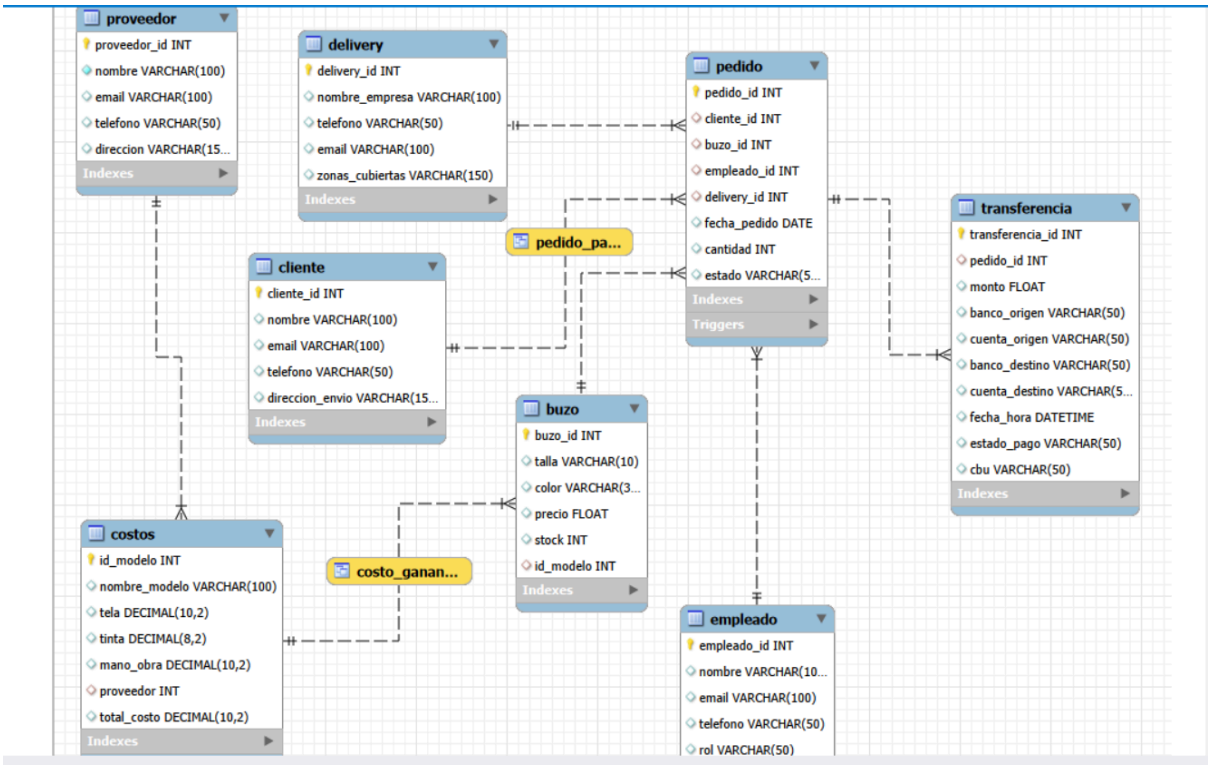
El modelo de negocio se desarrolla en varias etapas:

1. **Diseño y publicación de modelos:**
    - Se crean distintos modelos de buzos, cada uno con características específicas de diseño, estampado, color y talla.
    - Los modelos se publican en un catálogo o menú para que los clientes puedan seleccionar el estilo de estampado y personalización que desean.
2. **Selección y registro del pedido por parte del cliente:**
    - Una vez que el cliente selecciona el modelo deseado, se registra la información necesaria para la compra, incluyendo datos personales, dirección de envío y detalles del modelo elegido.
    - El cliente realiza la **transferencia previa** correspondiente al precio del producto, lo que permite garantizar la reserva del buzo.
3. **Producción y control de calidad:**
    - Los materiales para la producción de los buzos se obtienen a través de proveedores especializados, asegurando la calidad del textil y los insumos de estampación.
    - Los empleados del emprendimiento se encargan de la estampación y confección del buzo siguiendo las especificaciones del modelo y del cliente.
    - La producción se gestiona de manera que se optimice el stock disponible, se controlen los costos de materiales, tinta y mano de obra, y se mantenga un registro de cada modelo fabricado mediante la base de datos.
4. **Entrega del producto:**
    - Una vez que el buzo está listo, se organiza la entrega utilizando empresas de **delivery** asociadas o mediante puntos de encuentro establecidos con el cliente.

    - La información del envío, incluyendo empresa de delivery, fecha y estado del pedido, se almacena en la base de datos para asegurar el seguimiento del pedido y la satisfacción del cliente.
    - El cliente tiene la opción de retirar personalmente el producto en caso de preferirlo, lo cual también queda registrado en el sistema para efectos de control de stock y trazabilidad.
5. **Gestión de información y análisis:**
    - La base de datos permite relacionar clientes, pedidos, buzos, proveedores, empleados, transferencias y delivery, garantizando integridad y consistencia de la información.
    - Esto habilita la generación de informes que facilitan decisiones sobre rentabilidad de modelos, análisis de ventas, control de stock y seguimiento de pagos.

En síntesis, Cheers LK combina **personalización del producto**, **gestión de inventario y costos**, y **control del proceso de venta y entrega**, permitiendo ofrecer un servicio confiable y eficiente, además de posibilitar un análisis detallado de la información para mejorar la toma de decisiones empresariales.

4. Diagrama Entidad-Relación. Generado por MySQL Workbench.



5. Descripción de tablas (y sus relaciones)

Tabla Buzo

Campo	Tipo de dato	Descripción
buzo_id	Entero	Identificador único del buzo (PK)
nombre_modelo	Cadena de caracteres	Nombre del modelo del buzo
talla	Cadena de caracteres	Talla del buzo
color	Cadena de caracteres	Color del buzo
precio	Decimal	Precio de venta del buzo
stock	Entero	Cantidad disponible
proveedor_id	Entero	Relación con proveedor (FK)
id_modelo	Entero	Relación con la tabla costos (FK)

Funcionalidad: Almacena los productos que ofrece la empresa.
Relaciones:

- Muchos a uno con proveedores.
- Muchos a muchos con pedidos (implementado desde la tabla pedidos).

Tabla Proveedores

Campo	Tipo de dato	Descripción
proveedor_id	Entero	Identificador único (PK)
nombre	Cadena de caracteres	Nombre del proveedor
email	Cadena de caracteres	Correo electrónico
teléfono	Cadena de caracteres	Teléfono
dirección	Cadena de caracteres	Dirección

Funcionalidad: Guarda datos de proveedores.
Relaciones: Uno a muchos con buzos.

Tabla Clientes

Campo	Tipo de dato	Descripción
cliente_id	Entero	Identificador único (PK)
nombre	Cadena de caracteres	Nombre del cliente
email	Cadena de caracteres	Correo electrónico
teléfono	Cadena de caracteres	Teléfono
dirección_envío	Cadena de caracteres	Dirección de envío

Funcionalidad: Almacena información de clientes.

Relaciones: Uno a muchos con pedidos.

Tabla empleados:

Campo	Tipo de dato	Descripción
empleado_id	Entero	Identificador único (PK)
nombre	Cadena de caracteres	Nombre del empleado
email	Cadena de caracteres	Correo electrónico
teléfono	Cadena de caracteres	Teléfono
rol	Cadena de caracteres	Rol en la empresa

Funcionalidad: Empleados encargados de la estampación.

Relaciones: Uno a muchos con pedidos.

Tabla Delivery

Campo	Tipo de dato	Descripción
delivery_id	Entero	Identificador único (PK)
nombre_empresa	Cadena de caracteres	Nombre de la empresa
teléfono	Cadena de caracteres	Teléfono
email	Cadena de caracteres	Correo electrónico
zona_cubierta	Cadena de caracteres	Zona geográfica de cobertura

Funcionalidad: Almacena empresas de delivery.

Relaciones: Uno a muchos con pedidos.

Tabla Pedidos

Campo	Tipo de dato	Descripción
pedido_id	Entero	Identificador único del pedido (PK)
cliente_id	Entero	Relación con clientes (FK)
buzo_id	Entero	Relación con buzos (FK)
empleado_id	Entero	Relación con empleados (FK)
delivery_id	Entero	Relación con delivery (FK)
fecha_pedido	Fecha	Fecha del pedido
cantidad	Entero	Cantidad de buzos pedidos
estado	Cadena de caracteres	Estado del pedido

Funcionalidad: Registra cada pedido realizado.
Relaciones:

- Muchos a uno con clientes, empleados, delivery y buzos.
- Uno a uno con transferencias.

Tabla Transferencias

Campo	Tipo de dato	Descripción
transferencia_id	Entero	Identificador único (PK)
pedido_id	Entero	Relación con pedidos (FK)
monto	Decimal	Monto transferido
banco_origen	Cadena de caracteres	Banco de origen
cuenta_origen	Cadena de caracteres	Cuenta de origen
banco_destino	Cadena de caracteres	Banco receptor
cuenta_destino	Cadena de caracteres	Cuenta receptora
fecha_hora	Fecha y hora	Fecha y hora de la transferencia
estado_pago	Cadena de caracteres	Estado del pago
CBU	Cadena de caracteres	CBU de destino

Funcionalidad: Registra pagos de los pedidos.
Relaciones: Uno a uno con pedidos.

Tabla Costos

Campo	Tipo de dato	Descripción
id_modelo	Entero	Identificador del modelo (PK)
nombre_modelo	Cadena de caracteres	Nombre del modelo
costo_tela	Decimal	Costo de tela
costo_tinta	Decimal	Costo de tinta
mano_obra	Decimal	Costo de mano de obra
proveedor_id	Entero	Relación con proveedor (FK)
costo_total	Decimal	Costo total de producción

Funcionalidad: Almacena costos de producción de cada modelo.

Relaciones: Uno a muchos con buzos.

6. Inserción de datos de prueba

Se insertaron **10 registros de prueba** en un inicio, en cada tabla principal para verificar la funcionalidad del sistema:

- Proveedor, Buzo, Cliente, Empleado, Delivery, Pedido y Transferencia.

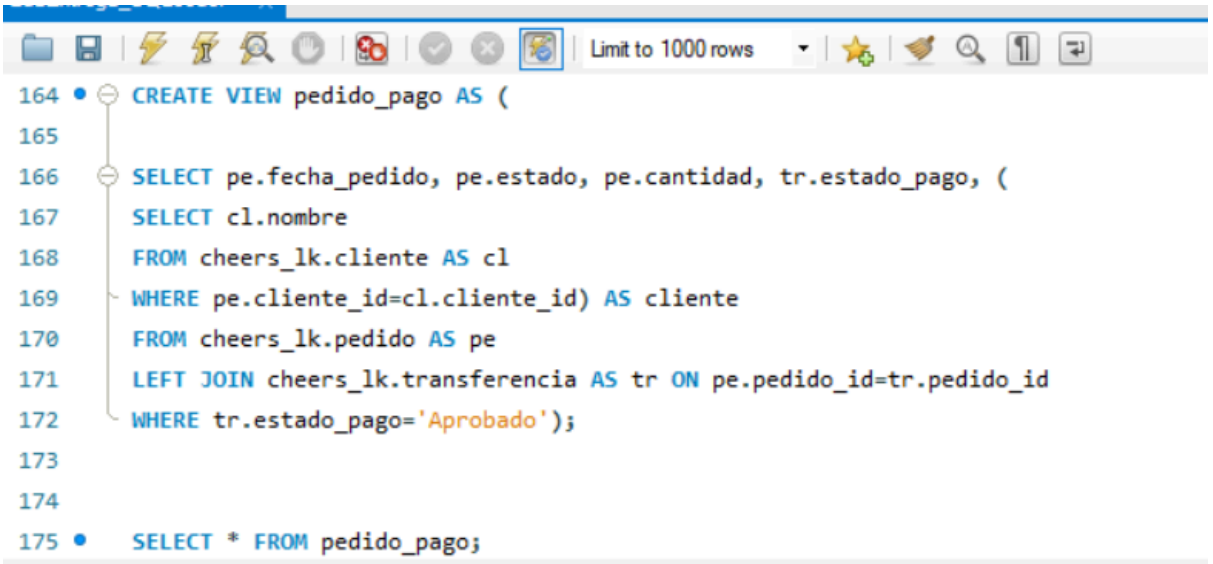
Se utilizó la sentencia INSERT INTO en MySQL Workbench y se realizaron pruebas de integridad y consistencia de datos.

Posteriormente se agregaron más datos para la consistencia en el análisis de datos con Power BI.

7. Vistas

Vista: pedido_pago

Descripción: Muestra los pedidos con pagos aprobados por transferencia.
Objetivo: Facilitar seguimiento de pagos y preparación de envíos.
Tablas utilizadas: Pedido, Transferencia, Cliente.
Screenshot:



Vista: `costo_ganancia`

Descripción: Relaciona el costo de producción de un modelo con el precio de venta.
Incluye el total de pedidos entregados por modelo.
Objetivo: Evaluar rentabilidad por modelo.
Tablas utilizadas: Buzos, Costos, Pedido.
Mejora implementada: Se ajustó la subquery para sumar solo los pedidos entregados correspondientes al mismo modelo de buzo, utilizando `START TRANSACTION` y `COMMIT`.
Screenshot:

```sql
CREATE VIEW cheers_lk.costo_ganancia AS (

SELECT
cs.total_costo,
cs.nombre_modelo,
SUM(bu.precio) AS total_precio_buzos,
(
    SELECT SUM(pe.cantidad)
    FROM cheers_lk.pedido AS pe
    WHERE pe.estado = 'Entregado'
) AS total_pedidos_entregados
FROM cheers_lk.buzo AS bu
LEFT JOIN cheers_lk.costos AS cs
ON bu.id_modelo = cs.id_modelo
GROUP BY cs.id_modelo, cs.total_costo, cs.nombre_modelo
);

SELECT * FROM cheers_lk.costo_ganancia;
```

8. Funciones y procedimientos

Función: `ganancia_por_modelo`

- **Descripción:** Calcula ganancia promedio por modelo: `precio promedio - costo total`.
- **Objetivo:** Análisis rápido de rentabilidad.
- **Tablas utilizadas:** Buzos, Costos.

```sql
CREATE FUNCTION ganancia_por_modelo (id INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN

DECLARE ganancia DECIMAL(10,2);

SELECT AVG(bu.precio) - co.total_costo
INTO ganancia
FROM cheers_lk.buzo AS bu
JOIN cheers_lk.costos AS co
ON bu.id_modelo = co.id_modelo
WHERE co.id_modelo = id
```

```sql
SELECT AVG(bu.precio) - co.total_costo
INTO ganancia
FROM cheers_lk.buzo AS bu
JOIN cheers_lk.costos AS co
ON bu.id_modelo = co.id_modelo
WHERE co.id_modelo = id
GROUP BY co.total_costo;

RETURN ganancia;

END //

DELIMITER ;
```

```sql
SELECT ganancia_por_modelo(1);
```

Procedimiento: `pedidos_por_cliente`

- **Descripción:** Permite consultar todos los pedidos de un cliente específico.
- **Objetivo:** Simplificar consultas de historial de pedidos.
- **Tablas utilizadas:** Pedido, Cliente.

```
USE cheers_lk

DELIMITER //

CREATE PROCEDURE pedidos_por_cliente(IN id_cliente INT)
BEGIN

SELECT
pe.pedido_id,
cl.nombre AS cliente,
pe.buzo_id,
pe.fecha_pedido,
pe.cantidad,
```

```
cl.nombre AS cliente,
pe.buzo_id,
pe.fecha_pedido,
pe.cantidad,
pe.estado
FROM pedido AS pe
JOIN cliente AS cl
ON pe.cliente_id = cl.cliente_id
WHERE pe.cliente_id = id_cliente;

END //

DELIMITER ;

CALL pedidos_por_cliente(8);
```

9. Triggers

Trigger: `actualizar_stock`

- **Descripción:** Se ejecuta al insertar un pedido, reduciendo automáticamente el stock del producto.
- **Tablas afectadas:** Pedido, Buzos.
- **Beneficio:** Evita actualizaciones manuales de stock.

```
    TRIGGER
=================================================== */

DELIMITER //

CREATE TRIGGER actualizar_stock
AFTER INSERT ON pedido
FOR EACH ROW
BEGIN

UPDATE buzo
SET stock = stock - NEW.cantidad
WHERE buzo_id = NEW.buzo_id;costo_ganancia

END DELIMITER //
```

10. Informes generados

En el desarrollo de la base de datos se implementaron distintos mecanismos de consulta e interpretación de datos con el objetivo de generar **informes útiles para la toma de decisiones del emprendimiento**. Estos informes permiten analizar tanto el comportamiento de las ventas como la rentabilidad de los productos y la interacción con los clientes.

10.1 Mejora en la vista `costo_ganancia`

Uno de los principales informes desarrollados corresponde a la vista `costo_ganancia`, cuyo objetivo es analizar la relación entre el costo de producción y el precio de venta de los buzos, incorporando además el total de unidades vendidas por modelo.

Inicialmente, esta vista presentaba una limitación importante: la columna correspondiente a **total de pedidos entregados** mostraba un valor constante para todos los modelos, lo cual impedía realizar un análisis correcto de la demanda por producto. Esto ocurría porque la subconsulta no estaba correctamente vinculada al modelo de buzo, sino que realizaba un cálculo global.

A partir de este problema, se implementó una mejora en la lógica de la subquery:

- Se incorporó un **JOIN entre las tablas `pedido` y `buzo`**, permitiendo relacionar cada pedido con su modelo correspondiente.
- Se agregó una condición en el `WHERE` que vincula el `id_modelo` del buzo con el `id_modelo` de la tabla `costos`.
- Se mantuvo el filtro por estado del pedido (`Entregado`) para asegurar que solo se contabilicen ventas efectivamente concretadas.

Gracias a esta modificación, el campo **total_pedidos_entregados** pasó a reflejar correctamente la cantidad de unidades vendidas por cada modelo, alineándose con el nivel de agrupación de la vista.

Durante el proceso de mejora se utilizaron herramientas de control de transacciones como:

- `START TRANSACTION`
- `ROLLBACK`
- `COMMIT`
- `SET autocommit = 0`

Estas permitieron realizar pruebas sin afectar permanentemente los datos hasta verificar el correcto funcionamiento de la consulta.

Esta mejora resulta clave para el negocio, ya que permite:

- Analizar qué modelos tienen mayor salida.
- Evaluar la rentabilidad en función del volumen de ventas.
- Tomar decisiones estratégicas sobre producción y precios.

10.2 Informe de clientes mediante el procedimiento `pedidos_por_cliente`

Otro informe relevante se genera a través del procedimiento almacenado **`pedidos_por_cliente`**, el cual permite consultar de manera sencilla todos los pedidos realizados por un cliente específico.

Este procedimiento devuelve información como:

- ID del pedido
- Nombre del cliente
- Producto adquirido
- Fecha del pedido
- Cantidad
- Estado del pedido

El principal valor de este informe radica en la posibilidad de:

- Identificar clientes frecuentes.
- Analizar el historial de compras.
- Detectar patrones de consumo.

Desde el punto de vista del negocio, esta información puede utilizarse para:

- Implementar estrategias de fidelización.

- Ofrecer promociones personalizadas.
- Realizar sorteos o beneficios para clientes recurrentes.

10.3 Aplicación de herramientas de Business Intelligence (Power BI)

En línea con los conceptos vistos en el módulo de **Data Warehouse y herramientas de Business Intelligence**, la información generada por la base de datos puede ser utilizada para la construcción de dashboards y visualizaciones en **Power BI**.

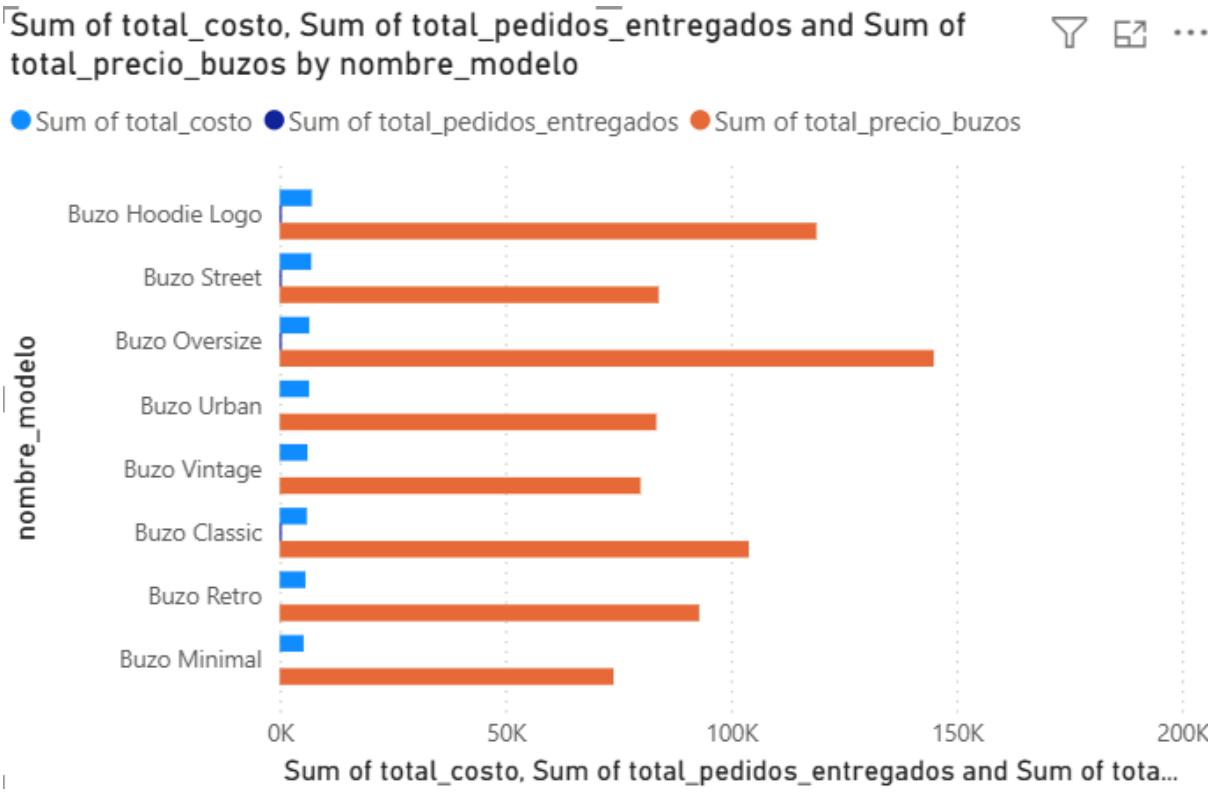
Las vistas desarrolladas, especialmente `costo_ganancia`, funcionan como una fuente ideal para este tipo de herramientas, ya que presentan datos ya procesados y listos para análisis.

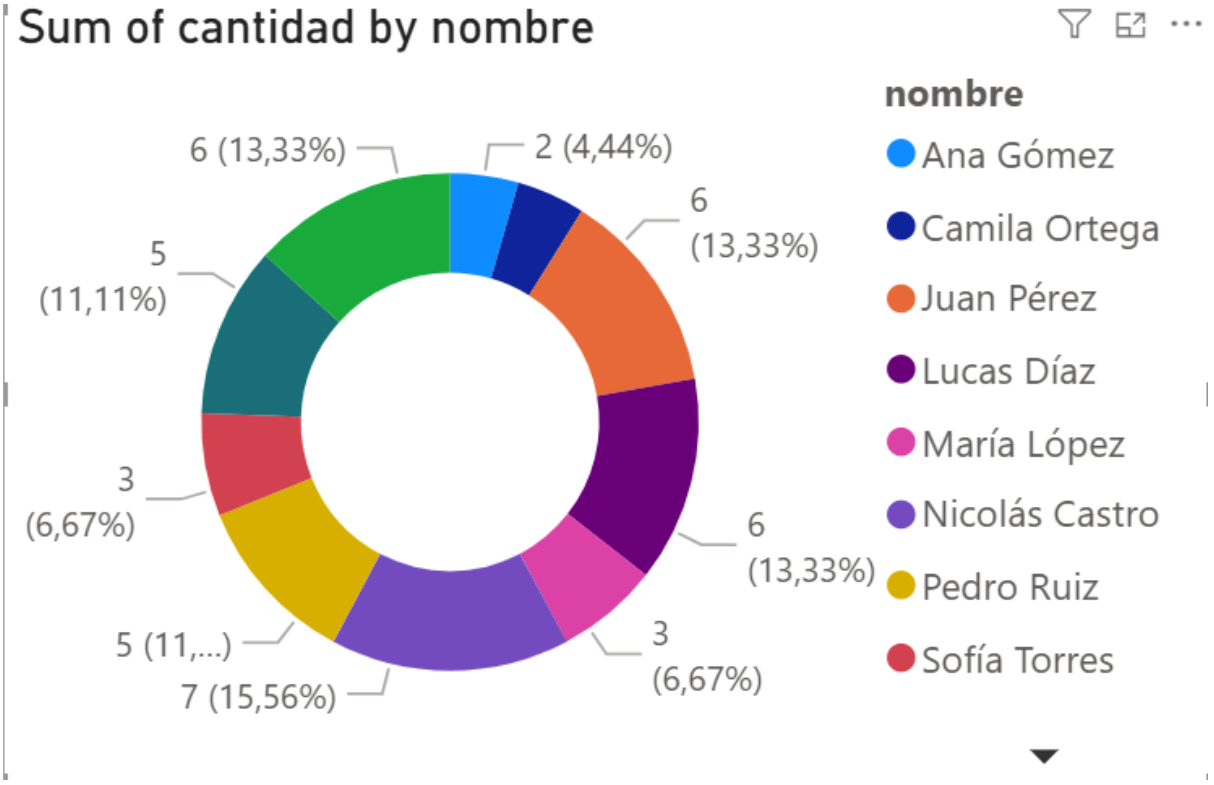
A partir de estas vistas, es posible generar visualizaciones como:

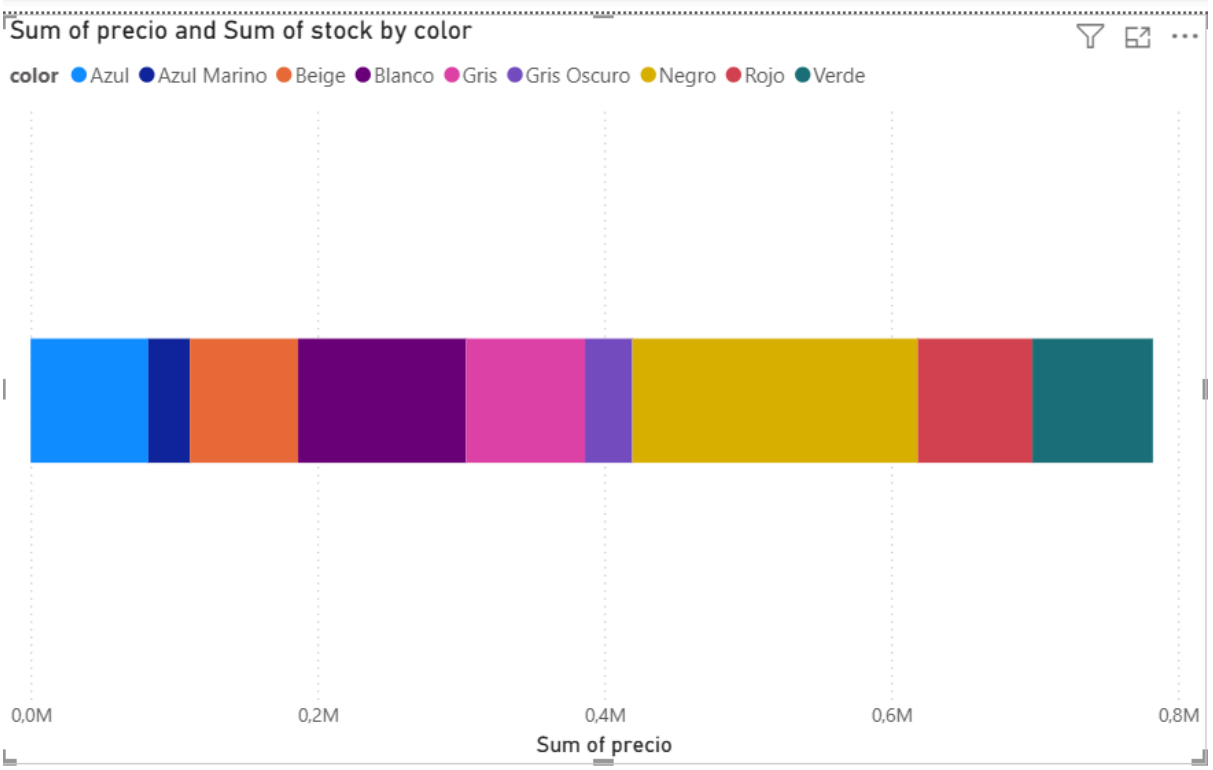
- Gráficos de barras comparando **costos vs ingresos por modelo**.
- Gráficos de columnas mostrando **cantidad de unidades vendidas por modelo**.
- Indicadores de rentabilidad por producto.

Estas visualizaciones permiten:

- Comprender rápidamente el rendimiento del negocio.
- Detectar oportunidades de mejora.
- Facilitar la toma de decisiones basada en datos.


Sum of total_costo, Sum of total_pedidos_entregados and Sum of total_precio_buzos by nombre_modelo
Sum of total_costo
Sum of total_pedidos_entregados
Sum of total_precio_buzos
nombre_modelo
Buzo Hoodie Logo
Buzo Street
Buzo Oversize
Buzo Urban
Buzo Vintage
Buzo Classic
Buzo Retro
Buzo Minimal
0K
50K
100K
150K
200K
Sum of total_costo, Sum of total_pedidos_entregados and Sum of tota...


Sum of cantidad by nombre
6 (13,33%)
2 (4,44%)
6 (13,33%)
5 (11,11%)
3 (6,67%)
6 (13,33%)
5 (11,...)
7 (15,56%)
3 (6,67%)
nombre
Ana Gómez
Camila Ortega
Juan Pérez
Lucas Díaz
María López
Nicolás Castro
Pedro Ruiz
Sofía Torres



10.4 Conclusión de los informes

Los informes generados a partir de la base de datos permiten transformar los datos operativos en **información estratégica**, aportando valor al emprendimiento.

La combinación de:

- consultas SQL optimizadas,
- vistas diseñadas para análisis,
- procedimientos almacenados,
- y herramientas de visualización como Power BI

permite construir un sistema que no solo registra información, sino que también la convierte en conocimiento útil para la gestión del negocio.

11. Orden de ejecución

Primer script a ejecutar: script_creacion

1. Creación de la base de datos
En primer lugar, se debe crear la base de datos utilizando la sentencia:

```
CREATE DATABASE IF NOT EXISTS cheers_lk;
```

El uso de *IF NOT EXISTS* permite evitar errores en caso de que la base ya haya sido creada previamente.

Luego, se debe establecer el contexto de trabajo mediante:

```
USE cheers_lk;
```

Esto garantiza que todas las operaciones posteriores se ejecuten dentro de esta base de datos.

---

2. Creación de tablas
A continuación, se procede a la creación de todas las tablas necesarias, también utilizando *IF NOT EXISTS*.

Es fundamental respetar el **orden de dependencia entre tablas**:

- Primero se crean las tablas independientes (por ejemplo: proveedor, cliente, empleado, delivery).
- Luego se crean las tablas dependientes que contienen claves foráneas (por ejemplo: buzo, pedido, transferencia).

Esto se debe a que MySQL no permite crear una clave foránea que haga referencia a una tabla que aún no existe.

---

3. Modificaciones estructurales (ALTER TABLE)
Una vez creadas las tablas, se deben ejecutar todas las sentencias de **ALTER TABLE**.

Estas modificaciones pueden incluir:

- Agregado de nuevas columnas (por ejemplo, `id_modelo` en buzo).
- Eliminación de columnas que ya no se utilizarán.
- Incorporación de nuevas claves foráneas.
- Ajustes en relaciones entre tablas.

Este paso debe realizarse antes de insertar datos, ya que la estructura final de las tablas debe estar completamente definida.

6. Creación de vistas
Una vez cargados los datos, se pueden crear las **vistas**.

Las vistas dependen de:

- La existencia de las tablas.
- La disponibilidad de datos.

Por lo tanto, deben crearse después de las inserciones.

---

7. Creación de funciones
En este paso se crean las **funciones definidas por el usuario**, como por ejemplo el cálculo de ganancia por modelo.

Estas funciones dependen de:

- Tablas existentes.
- Relaciones correctamente definidas.

---

8. Creación de procedimientos almacenados
Luego se crean los **stored procedures**, que permiten encapsular consultas complejas.

Ejemplo: consulta de pedidos por cliente.

Al igual que las funciones, dependen de la estructura y los datos previamente cargados.

---

9. Creación de triggers
Finalmente, se crean los **triggers**.

Estos se ejecutan automáticamente ante eventos (por ejemplo, insertar un pedido) y dependen de:

- La existencia de las tablas involucradas.
- La lógica de negocio ya definida.

Por ejemplo, el trigger de actualización de stock requiere que las tablas pedido y buzo ya existan y estén correctamente relacionadas.

Segundo script a ejecutar: script_insercion

4. Actualización de datos existentes (UPDATE)
Luego de las modificaciones estructurales, se realizan las sentencias de **UPDATE** necesarias.

Estas actualizaciones permiten:

- Adaptar registros existentes a nuevas columnas.
- Establecer relaciones (por ejemplo, asignar `id_modelo` a los buzos).

Es importante que este paso sea posterior a los ALTER TABLE, ya que depende de los cambios realizados en la estructura.

---

5. Inserción de datos (INSERT INTO)
Una vez que la estructura está completamente definida y actualizada, se procede a la carga de datos.

Se debe respetar nuevamente el orden de dependencia:

- Primero se insertan datos en tablas independientes (proveedor, cliente, empleado, delivery).
- Luego en tablas relacionadas (buzo, pedido, transferencia).

Además, los datos deben ser coherentes con las claves foráneas y las restricciones definidas previamente.

---

12. Herramientas y Tecnología utilizada.

MySQL Workbench: Desarrollo y pruebas en la base de datos.

Power BI: Visualización y monitoreo de datos(gráficos para analizar).

ChatGPT: Para la organización prolija del código, la inserción de datos en las tablas, y la prolijidad del informe.

13. Repositorios y links de acceso:

https://github.com/moracoghlan/ProyectoCoderSQL